EXPERIMENT No 3:

Building a line Following Robot Using PID Control

Student Guide

Prepared by BLIS GLOBAL Robotics Department

Copyright @2024

Objective

In this project, you will learn how to create a line-following robot using 3-line tracking sensors, an L293D motor shield (with M1, M2, M3, and M4 connections), and implement a PID (Proportional-Integral-Derivative) control system in Arduino to enhance the robot's accuracy in following a line.

Materials Required

- Arduino Uno R3
- L293D motor shield (with M1, M2, M3, M4 motor connections)
- 4 DC motors (for 4WD)
- 3 Line tracking sensors (IR sensors)
- Jumper wires
- 4 wheels and chassis
- 12V battery or equivalent power source
- Arduino IDE installed on your computer

Understanding PID Control

A PID controller is used to control the speed and direction of the motors for smoother line following. Here's a brief overview of the components of PID:

- Proportional (P): Determines the robot's reaction to the current position error.
- Integral (I): Accounts for the past errors to correct drift.
- Derivative (D): Predicts future errors by observing the rate of change.

By adjusting these values (Kp, Ki, Kd), you can tune the performance of the robot.

Wiring Diagram

1. L293D Motor Shield:
    - M1 and M2: Connect to the left-side motors.
    - M3 and M4: Connect to the right-side motors.
2. Line Tracking Sensors:
    - Place the 3-line tracking sensors at the front of the robot.
    - Connect each sensor's VCC to 5V, GND to GND, and the OUT pins to Arduino digital pins (e.g., pins 2, 3, 4).

Step-by-Step Instructions

1. **Assemble the Robot**

- Mount the motors and wheels on the chassis.
- Attach the L293D motor shield on top of the Arduino.
- Connect the motors to the appropriate terminals on the motor shield (M1, M2, M3, M4).
- Position the line tracking sensors in the front of the robot. Ensure that they are aligned to detect the black line on the white surface.

2. **Connect the Components**

- Connect the outputs of the 3-line tracking sensors to digital pins on the Arduino (e.g., sensor 1 to pin 2, sensor 2 to pin 3, sensor 3 to pin 4).
- Power the motor shield and sensors using a 12V battery connected to the motor shield.

3. Arduino Code

Here's an example code implementing PID control for the line-following robot. You'll need to fine-tune the Kp, Ki, and Kd values to get the best performance.

```
#include <AFMotor.h>
// Initialize motors
AF_DCMotor motor1(1);  // Left motors (M1)
AF_DCMotor motor2(2);  // Left motors (M2)
AF_DCMotor motor3(3);  // Right motors (M3)
AF_DCMotor motor4(4);  // Right motors (M4)

// PID parameters
float Kp = 1.5;  // Proportional
float Ki = 0;    // Integral
float Kd = 0.5;  // Derivative

int lastError = 0;
int integral = 0;

// Line sensor pins
int sensorLeft = 2;
int sensorCenter = 3;
int sensorRight = 4;
```

```
// Motor speed variables
int baseSpeed = 150;   // Base speed for motors
int maxSpeed = 255;    // Max motor speed

void setup() {
  pinMode(sensorLeft, INPUT);
  pinMode(sensorCenter, INPUT);
  pinMode(sensorRight, INPUT);

  Serial.begin(9600);
}

void loop() {
  // Read sensor values
  int leftSensor = digitalRead(sensorLeft);
  int centerSensor = digitalRead(sensorCenter);
  int rightSensor = digitalRead(sensorRight);
```

```
  // Calculate the error (Position)
  int error = (leftSensor * -1) + (centerSensor * 0) +
(rightSensor * 1);

  // PID calculations
  integral = integral + error;
  int derivative = error - lastError;
  int output = (Kp * error) + (Ki * integral) + (Kd * derivative);
  lastError = error;

  // Adjust motor speeds
  int leftMotorSpeed = baseSpeed - output;
  int rightMotorSpeed = baseSpeed + output;

  // Constrain motor speeds to max and min
  leftMotorSpeed = constrain(leftMotorSpeed, 0, maxSpeed);
  rightMotorSpeed = constrain(rightMotorSpeed, 0,
maxSpeed);

  // Set motor speeds
```

```
  motor1.setSpeed(leftMotorSpeed);
  motor2.setSpeed(leftMotorSpeed);
  motor3.setSpeed(rightMotorSpeed);
  motor4.setSpeed(rightMotorSpeed);

  motor1.run(FORWARD);
  motor2.run(FORWARD);
  motor3.run(FORWARD);
  motor4.run(FORWARD);
}
```

4. **Tuning the PID**

- Kp (Proportional): Increase the Kp value if the robot is not turning fast enough, and decrease it if it oscillates too much.
- Ki (Integral): Adjust Ki to correct any drift in the robot's path.
- Kd (Derivative): Use Kd to smooth out sharp changes in direction.

Start with Kp only, then slowly add Ki and Kd as needed to get smooth line following.

5. **Test the Robot**

- Place the robot on a track with a black line on a white background.
- Upload the code to the Arduino.
- Power the robot and observe its behavior.
- Adjust the PID values as necessary to improve line tracking performance.

Key Learning Points

- Sensors: Understanding how line tracking sensors detect contrast between black and white surfaces.
- Motor Control: Using PWM to control motor speed and direction via the L293D motor shield.
- PID Tuning: Learning how PID control can be used to adjust the robot's movements to more accurately follow a line.

Troubleshooting Tips

- If the robot does not follow the line well, double-check the sensor connections and motor wiring.
- Adjust the Kp, Ki, and Kd values if the robot oscillates or reacts too slowly.
- Ensure that the line tracking sensors are properly aligned and detect the line clearly.

Additional Tasks

- Try to adjust the speed of the robot dynamically depending on the curves in the line.
- Experiment with different sensor placements to see how they affect the robot's performance.

By completing this project, you will have a solid understanding of how to build and program a line-following robot using PID control!